



COLLEGE OF ENGINEERING AND TECHNOLOGY(COETEC)

SCHOOL OF ELECTRICAL, ELECTRONIC, AND INFORMATION ENGINEERING

B.Sc. TELECOMMUNICATION AND INFORMATION ENGINEERING

UNIT: DISTRIBUTED COMPUTING & APPLICATION

UNIT CODE: ICS 2403

TITLE: CAT 1

GROUP 1

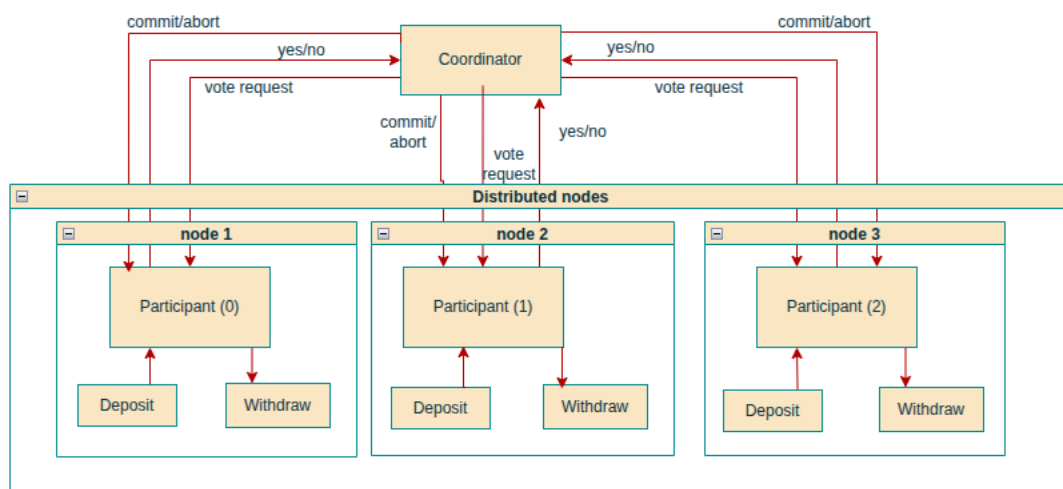
NO	NAME	REG NO
1	BRIDGETTE NTHUKA	ENE221-0097/2021
2	STANLEY OMONDI	ENE221-0140/2021
3	VICTOR OSIOLO	ENE221-0131/2021
4	CYNTHIA NJAMBI	ENE221-0117/2021
5	IRENE YEGON	ENE221-0128/2021
6	PEACE ISSA	ENE221-0132/2021

ATOMIX DISTRIBUTED TRANSACTION SYSTEM

INTRODUCTION

This project implements a small-scale distributed transaction system designed to maintain ACID (Atomicity, Consistency, Isolation, Durability) properties across multiple nodes. Using Python's multiprocessing framework, the system simulates a banking environment where users can transfer funds between distinct accounts. The solution integrates a Two-Phase Commit (2PC) protocol for atomicity, Strict Two-Phase Locking (S2PL) for concurrency control, and Write-Ahead Logging (WAL) for fault tolerance.

ARCHITECTURE DIAGRAM AND BRIEF DESCRIPTION OF THE SYSTEM



Our system shows how consistency is maintained in a bank system where we simulate money deposition, withdrawal and transfer between participants 0, 1 and 2 in different nodes maintaining concurrency, atomicity, and fault tolerance. Our system incorporates simulation of external systems integration such as Mpesa and Paypal for withdrawing and depositing money by each participant in the different nodes. 2PC ensures the transfer is either complete or total failure, no partiality. Our user interface is the CLI (command line interface)

The system follows a **Coordinator–Participant architecture** where the **coordinator** manages the lifecycle of transactions, initiates prepare and commit/abort phases, collects votes from participants and makes global commit or abort decisions. The **participants** maintain their own local databases (JSON files), maintain persistent transaction logs (WAL) and enforce locking and transaction rules. The **CLI Interface** allows users to submit transactions, supports failure injection and displays system state and invariants.

Distributed Nodes and Data Model

Each participant node runs in a **separate OS process**, has its own local storage under `dist_system_data/` and cannot directly access another node's memory or data ensuring **true distribution and fault isolation**, even though execution occurs on a single machine.

TRANSACTION MODEL

Transaction Type

The system implements a **money transfer transaction** between the different nodes.

Two-Phase Commit (2PC)

The system guarantees atomicity using the **Two-Phase Commit protocol**.

Phase 1: Prepare (Voting Phase)

1. Coordinator sends PREPARE to all nodes.
2. Each participant:
 - Checks for lock availability
 - Verifies business logic (e.g., sufficient funds)
 - Writes a PREPARED record to WAL
 - Votes **YES** or **NO**

Phase 2: Commit / Abort (Decision Phase)

- If **all participants vote YES** → COMMIT
- If **any participant votes NO or times out** → ABORT and the transaction is rolled back globally.

The coordinator then broadcasts the final decision to all participants.

CONCURRENCY STRATEGY

A threaded stress test generates random simultaneous transfers. The system successfully detects collisions and aborts conflicting transactions to preserve data integrity.

```
--- STARTING RANDOMIZED STRESS TEST ---
Generating 4 concurrent transactions...
[Thread-0] Requesting: Node 1 -> Node 2 ($1569)
[Coord] START TX-0: Node 1 -> Node 2 ($1569)
[Coord] FINISH TX-0: ABORT (Node 1 said INSUFFICIENT_FUNDS)
[Thread-1] Requesting: Node 2 -> Node 0 ($696)
[Coord] START TX-1: Node 2 -> Node 0 ($696)
[Coord] FINISH TX-1: COMMIT (Success)
[Thread-2] Requesting: Node 2 -> Node 0 ($531)
[Coord] START TX-2: Node 2 -> Node 0 ($531)
[Coord] FINISH TX-2: ABORT (Node 2 said INSUFFICIENT_FUNDS)
[Thread-3] Requesting: Node 2 -> Node 1 ($304)
[Coord] START TX-3: Node 2 -> Node 1 ($304)
[Coord] FINISH TX-3: COMMIT (Success)
--- STRESS TEST COMPLETE ---
```

By using Strict Two-Phase Locking (S2PL)

- Each participant maintains an in-memory exclusive lock per data item.
- Locks are acquired during PREPARE and released only after COMMIT or ABORT.

If a second transaction attempts to access a locked resource:

- The participant immediately votes **NO (LOCKED)**
- Coordinator aborts the transaction

FAILURE HANDLING AND RECOVERY APPROACH

These failures are representative of real-world distributed environments such as unreliable networks, overloaded services, or temporarily crashed nodes.

A "flaky mode" was introduced where specific nodes randomly sleep/delay during the PREPARE phase, simulating a network partition or crash:

- A participant node can be toggled into failure mode using the CLI (fail <node_id> on)
- When enabled, the node randomly delays its response during the PREPARE phase.
- The delay exceeds the Coordinator's timeout threshold, simulating a network partition or node crash.

Detection

During Phase 1 (Prepare to Vote), the Coordinator waits for votes from all participants. A **fixed timeout of 2 seconds** is enforced and if any participant fails to respond within this window, the Coordinator assumes failure. Failure detection is timeout-based.

Recovery Strategy and Consistency Preservation

The Coordinator unilaterally decides to **ABORT** the transaction and An ABORT message is broadcast to all participants who release any held locks and discard tentative changes.

This guarantees that:

- No partial state changes occur.
- Global invariants (total balance) are preserved.
- The system returns to a consistent state immediately after failure handling.

KEY OBSERVATIONS AND SYSTEM ANALYSIS

1. No transaction was ever partially applied
2. Even when a participant became unresponsive mid-transaction, the system consistently aborted the transaction
3. Account balances across all nodes always remained globally consistent.

4. Concurrent transactions were successful as long as the accounts had enough balance and their balances were correctly updated.

5. When a participant entered flaky mode, transaction latency increased due to timeout waits and the system temporarily rejected otherwise valid transactions.

TRADE-OFFS

Consistency vs Availability: When a failure occurred (Node 1 timeout), the system chose to abort the transaction rather than risk partial execution.

Strict Two-Phase Locking vs Optimistic Concurrency: Optimistic Concurrency Control (OCC) could improve throughput but would significantly complicate conflict detection and rollback logic.

CONCLUSION

The Atomix Distributed Transaction System successfully demonstrates fault-tolerant distributed transaction processing. By combining Two-Phase Commit, Strict Two-Phase Locking, and Write-Ahead Logging, the system ensures atomicity, isolation, and consistency even in the presence of node failures and concurrent access. Through controlled failure injection and systematic testing, the system clearly illustrates real-world distributed systems trade-offs, particularly the balance between consistency and availability showing its applicability in the banking sector.

REFERENCES

1. Gray, J. and Reuter, A. (1993), *Transaction Processing: Concepts and Techniques*. 1st ed. San Francisco: Morgan Kaufmann Publishers.
2. Bernstein, P. A., Hadzilacos, V. and Goodman, N. (1987), *Concurrency Control and Recovery in Database Systems*. 1st ed. Addison-Wesley.
3. Coulouris, G., Dollimore, J., and Kindberg, T. (2012), *Distributed Systems: Principles and Paradigms*. 2nd ed. Upper Saddle River, NJ: Prentice Hall.